CUDA Thread Hierarchy and GPU Architecture Notes

NVIDIA GPU learning notes

1. Grid and block scheduling

A **grid** is the collection of all thread blocks created by a single kernel launch. For example, launching a kernel with 8 x 8 blocks creates a grid of 64 blocks.

The GPU's **block scheduler** distributes these blocks across the Streaming Multiprocessors (SMs). Each block is assigned in full to one SM and is never split between SMs. For example, 4 blocks may be placed on SM 0 and 4 blocks on SM 1. Blocks are assigned dynamically: when a block finishes, its SM frees its resources and the scheduler gives it the next waiting block, so a grid larger than what the GPU can hold at once runs in waves.

2. Warps and CUDA cores

When a block is assigned to an SM, the SM divides its threads into groups of 32 consecutive threads called **warps**. A warp scheduler issues the same instruction to all 32 threads of a warp, and each thread applies it to its own data.

The arithmetic is carried out by **CUDA cores**, which are the SM's FP32 (single-precision floating-point) units. A 128-core SM contains 128 FP32 units, organized as 4 partitions of 32 units, and each partition has its own warp scheduler. Because a partition has 32 FP32 units, one warp's FP32 instruction runs on all 32 units in a single issue cycle, one thread per unit. The 4 partitions work in parallel, so up to 4 warps can issue an instruction in the same cycle.

On SMs with 16 FP32 units per partition (such as V100 and A100), the same warp instruction takes 2 cycles, with 16 threads executed per cycle.

Note: maximum threads per block. On all modern NVIDIA GPUs (compute capability 2.0 and later), a block can have at most **1024 threads**. The limit applies to the product of the dimensions: blockDim.x * blockDim.y * blockDim.z must not exceed 1024. Each dimension also has its own limit: x up to 1024, y up to 1024, and z up to 64. For example, dim3(32, 32, 1) and dim3(1024, 1, 1) are valid, while dim3(16, 16, 8) is invalid because it gives 2048 threads. A launch that breaks a limit fails with the error 'invalid configuration argument'. This is a maximum, not a typical size: any size from 1 to 1024 is allowed, and a multiple of 32 between 128 and 512 is most common. It is also separate from the per-SM limit (1536 to 2048 resident threads shared by all blocks on that SM).

3. Warp schedulers

Each SM is divided into 4 partitions, and each partition has its own **warp scheduler**. The 4 schedulers in an SM run concurrently and independently. In every cycle, each scheduler looks at its own pool of resident warps, picks one that is ready, and issues one instruction to its own

partition's lanes. No scheduler waits for the others. Every SM on the GPU has its own set of 4 schedulers, so all of them operate at the same time across the chip.

What each partition owns: its own warp scheduler, register file, and execution units (for example 32 FP32 lanes, plus its share of INT32, load/store, and special function units, and one tensor core). A warp is assigned to one partition when its block lands on the SM, and that partition's scheduler issues all of its instructions.

What is shared: resources at the SM level, such as shared memory, the L1 cache, and the memory pipeline. If many warps issue loads at the same time, they can contend for these.

Independent issue: the 4 schedulers can issue different warps and even different instruction types in the same cycle. For example, partition 0 may issue an FP32 add while partition 1 issues a load and partition 2 issues an integer operation.

Example of one cycle on one SM: partitions 0, 1, and 3 each pick a ready warp and run it on 32 FP32 lanes, so 3 warps (96 threads) execute. Partition 2 has no ready warp because all of its warps are stalled, so its lanes sit idle for that cycle. This is why many resident warps matter: more candidates make it more likely that every scheduler finds a ready warp in every cycle.

4. Driver versus hardware: who does what

The NVIDIA driver prepares and submits the work, but the scheduling itself is done by the GPU hardware.

Done by the driver (software on the CPU):

• Allocates GPU memory and sets up page tables.
• Loads the kernel, JIT-compiling PTX to SASS if needed.
• Validates the launch configuration (for example, no more than 1024 threads per block).
• Writes a launch command into a queue (pushbuffer): kernel address, grid size, block size, arguments, and shared memory size.
• Tells the GPU to read that queue, then tracks completion and errors.

Done by the GPU hardware (no driver involved at run time):

• **Block scheduler** (NVIDIA's GigaThread engine): reads the launch command and assigns blocks to SMs as resources free up.
• Splits blocks into warps, and allocates registers and shared memory per block.
• **Warp schedulers** in each SM: pick a ready warp and issue instructions every cycle.
• Execution on the cores, including the thread ID registers (threadIdx, blockIdx).

Once the launch command reaches the GPU, the driver is no longer in the loop. Block and warp scheduling happen in silicon, making billions of decisions per second, which software on the CPU could not keep up with.

Recent NVIDIA GPUs also run a small firmware on an on-chip processor (the GSP) that handles some management work, such as power and context scheduling between processes.

It sits between the driver and the hardware but is not involved in per-block or per-warp scheduling.

Summary: the driver compiles, allocates, and submits the launch; the block scheduler (hardware) distributes blocks to SMs; the warp scheduler (hardware) issues instructions to the cores every cycle.

5. cudaDeviceSynchronize()

cudaDeviceSynchronize() makes the CPU wait until the GPU has finished all the work it was given. It blocks the calling CPU thread and returns only when every earlier kernel launch and memory operation on the device has completed.

Why it is needed: kernel launches are asynchronous. When the CPU executes VecAdd<<<blocks, threads>>>(...), it only places the launch command in a queue and moves on to the next line immediately, without waiting for the kernel to run.

```
VecAdd<<<blocks, threads>>>(A, B, C);
cudaDeviceSynchronize();   // CPU waits until the kernel is done
printf("%f\n", C[0]);      // result is now ready
```

When it is needed:

• **Unified memory (cudaMallocManaged):** the CPU reads results directly, so it must synchronize first.
• **Timing:** synchronize before stopping a CPU timer, otherwise only the launch cost is measured. (cudaEvent timers are usually better.)
• **Error checking:** errors that happen while a kernel runs, such as an out-of-bounds access, are reported only after execution, so check the return value of the sync call. A bad launch configuration is caught at launch with cudaGetLastError().
• **printf inside a kernel:** device output is flushed when the device synchronizes.

When it is not needed: a blocking cudaMemcpy already waits for earlier work in the same stream, so no sync is required before copying results back. Kernels launched in the same stream run in order, so a second kernel automatically waits for the first.

Cost and alternatives: it stalls the CPU and prevents it from overlapping its own work with the GPU, so avoid calling it after every kernel in performance-critical code. Finer-grained options are cudaStreamSynchronize(stream) to wait for one stream, cudaEventSynchronize(event) to wait for a specific point in a stream, and cudaMemcpyAsync with streams to overlap copies and kernels.

Not the same as __syncthreads(): __syncthreads() makes the threads of one block wait for each other inside a kernel, while cudaDeviceSynchronize() makes the CPU wait for the whole GPU from host code.

Link to section 4: the driver queues the launch and returns, the GPU hardware executes it, and cudaDeviceSynchronize() is how the CPU waits for the hardware to finish.

6. Memory

Device memory

The CUDA programming model assumes a system made of a **host** (the CPU) and a **device** (the GPU), each with its own separate memory. Kernels run out of device memory, so the CUDA runtime provides functions to allocate and free device memory, copy data within it, and transfer data between host memory and device memory (for example cudaMalloc, cudaFree, and cudaMemcpy).

Device memory can be allocated in two forms:

• **Linear memory:** a plain contiguous block of memory that a kernel addresses through ordinary pointers, like a normal array.
• **CUDA arrays:** opaque memory layouts optimized for texture fetching. They are not accessed through ordinary pointers, but through the texture and surface memory functions.

Source: NVIDIA CUDA C++ Programming Guide, section 6.2.2 (Device Memory).

Global memory

Global memory is the GPU's main memory (the on-board VRAM, built from GDDR or HBM). It is what cudaMalloc allocates. It is visible to **all threads of all blocks** and to the host (through cudaMemcpy), and its contents persist across kernel launches until you free it.

• **Large but slow:** capacity is many GB, but latency is high (hundreds of cycles). The L2 cache (and L1 on some architectures) reduces the cost of repeated accesses.
• **Coalescing matters:** when the 32 threads of a warp access consecutive addresses, the hardware merges them into a few wide transactions. Scattered accesses need many transactions and are much slower.
• **Visible between blocks:** it is the way separate blocks (and separate kernels) pass data to each other, using atomics or by launching another kernel.

```
float *d_A;
cudaMalloc(&d_A, N * sizeof(float));    // global memory
cudaMemcpy(d_A, h_A, N * sizeof(float),
           cudaMemcpyHostToDevice);    // host -> device
kernel<<<blocks, threads>>>(d_A);
cudaFree(d_A);
```

Thread-local memory

Each thread has its own private storage. The fastest form is **registers**, which live on-chip in the SM's register file. When a thread needs more private data than the registers can hold, the compiler places it in **local memory**.

• **Despite the name, local memory is not on-chip.** It is a private region of device DRAM, so it has the same high latency as global memory (it is cached in L1 and L2). "Local" refers to its scope (visible to one thread only), not its location.
• **What goes there:** registers that spill when a kernel uses too many, large per-thread arrays,

and arrays indexed with a value not known at compile time.

• **The compiler decides automatically.** You do not declare it. To check for spills, compile with --ptxas-options=-v, which reports spill stores and loads.

• **Cost:** heavy local memory use slows a kernel. Reduce register pressure, shrink per-thread arrays, or move shared data into shared memory.

```
__global__ void k(int idx) {
    float x = 1.0f;          // register (fast, on-chip)
    float buf[64];           // likely local memory if indexed dynamically
    buf[idx] = x;            // idx unknown at compile time
}
```

Unified memory

Unified memory (managed memory) gives the CPU and the GPU a **single pointer** that works on both sides, so you do not copy data back and forth with cudaMemcpy. It is allocated with cudaMallocManaged.

• **How it works:** the driver migrates memory pages on demand between system RAM and GPU memory, usually triggered by page faults when one side touches data that currently lives on the other.

• **Synchronize before the CPU reads:** kernel launches are asynchronous, so call cudaDeviceSynchronize() before the CPU accesses results.

• **Tuning:** cudaMemPrefetchAsync moves data ahead of time, and cudaMemAdvise gives placement hints. Both avoid page-fault delays.

• **Larger than GPU memory:** on supported systems, managed memory can exceed the GPU's VRAM by paging, at a cost in speed.

• **Hardware matters:** on discrete GPUs, data still moves over PCIe, so naive use can be slower than explicit copies. On systems where the CPU and GPU share physical memory (Apple Silicon, Jetson, Grace Hopper), no copying is needed at all.

```
float *data;
cudaMallocManaged(&data, N * sizeof(float));  // one pointer
for (int i = 0; i < N; ++i) data[i] = i;      // CPU writes
kernel<<<blocks, threads>>>(data, N);         // GPU, same pointer
cudaDeviceSynchronize();                      // wait before CPU reads
printf("%f\n", data[0]);
```

Summary of memory types

Memory	Visible to	Location	Notes
Registers	One thread	On-chip (SM register file)	Fastest; limited per thread
Local memory	One thread	Off-chip (device DRAM)	Spills and large arrays; as slow as global
Shared memory	One block	On-chip (SM)	Fast; used to share data inside a block
Global memory	All threads and the host	Off-chip (device DRAM)	Large; high latency; coalesce accesses
Unified (managed)	CPU and GPU	Migrates between host RAM and VRAM	One pointer; simpler code; page migration cost